

CS 4910 Lab 2 Report

Packet Sniffing and Spoofing Lab

Notes: (IMPORTANT)

- It is required to use this report template.
- Report your work in each section. **Describe** what you have done and what you have observed. You should take screenshots to support your description. You also need to provide an explanation of the observations that are interesting or surprising. Please also list the important code snippets followed by an explanation.
- Simply attaching code or screenshots without any explanation will NOT receive credits.
- **Do NOT claim anything you didn't do.** If you didn't try on a certain task, leave that section blank. You will receive a ZERO for the whole assignment if we find any overclaim.
- Grading will be based on your **description** and the completion of each task.
- After you finish, export this report as a PDF file and submit it on Canvas.

Your Full Name: Caroline Marie Duncan

Email Address: cduncan2@uccs.edu

I, Caroline Duncan(Your Full Name), have read and understood the course academic integrity policy.

(Your report will not be graded without filling in the above AI statement.)

1. Task Set 1: Using Scapy to Sniff and Spoof Packets

Task 1.1. Sniffing Packets

Task 1.1A

Quick Note: Since my Ubuntu VM would not boot on my Macbook, I completed this Lab on a AWS EC2 instance, docker, scapy and tcpdump were installed on the instance and the SEED Sniffing/Spoofing Labsetup containers were brought up with docker-compose.

The bridge interface created by Docker for the 10.9.0.0/24 network was br-aeaa556e9e85, which is the interface I sniffed on for every task. tcpdump was used instead of Wireshark since the instance is headless.

I wrote a Python sniffer using Scapy that captures packets on the Docker bridge interface (br-aeaa556e9e85) and prints the source IP, destination IP and protocol number of each captured packet.

The filter was set to 'icmp' so only ICMP traffic is shown. To generate ICMP traffic I opened a shell on host A and pinged host B using ping 10.9.0.6.

A screenshot of a terminal window showing the nano 6.2 text editor editing a file named sniffer.py. The script content is as follows:

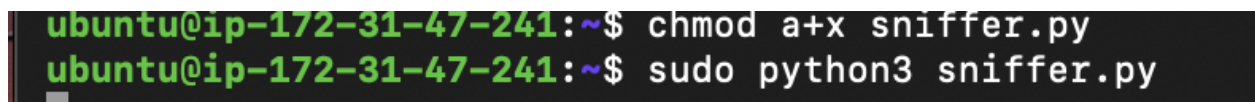
```
#!/usr/bin/env python3
from scapy.all import *

def print_pkt(pkt):
    print("Source IP:      ", pkt[IP].src)
    print("Destination IP:  ", pkt[IP].dst)
    print("Protocol:         ", pkt[IP].proto)
    print("_" * 40)

pkt = sniff(iface='br-aeaa556e9e85', filter='icmp', prn=print_pkt)
```

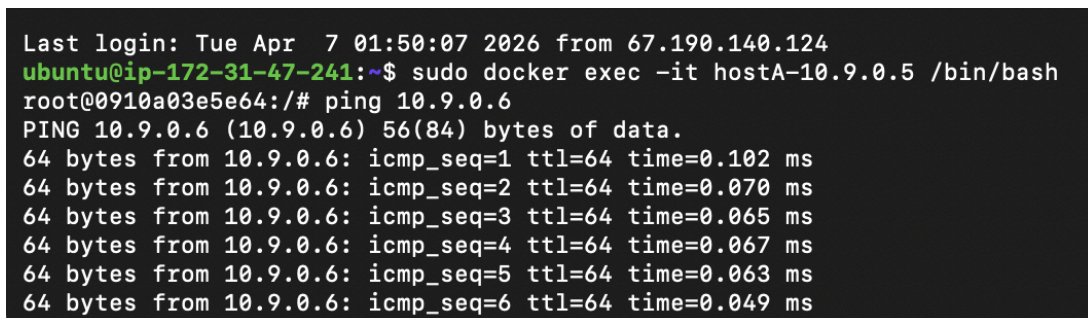
Photo #1: Sniffer.py script

Running the script with sudo, I captured ICMP echo requests and the corresponding echo replies. Protocol 1 in the output corresponds to ICMP.

A screenshot of a terminal window showing two commands being executed on a system with IP 172.31.47.241. The first command is 'chmod a+x sniffer.py' and the second is 'sudo python3 sniffer.py'.

```
ubuntu@ip-172-31-47-241:~$ chmod a+x sniffer.py
ubuntu@ip-172-31-47-241:~$ sudo python3 sniffer.py
```

Photo #2: Using sudo to run the script

A screenshot of a terminal window showing the output of the sniffer script. It shows a shell on host A (root@00910a03e5e64) pinging 10.9.0.6. The output displays six ICMP echo requests and replies with their sequence numbers, TTLs, and round-trip times.

```
Last login: Tue Apr  7 01:50:07 2026 from 67.190.140.124
ubuntu@ip-172-31-47-241:~$ sudo docker exec -it hostA-10.9.0.5 /bin/bash
root@00910a03e5e64:/# ping 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data.
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=0.102 ms
64 bytes from 10.9.0.6: icmp_seq=2 ttl=64 time=0.070 ms
64 bytes from 10.9.0.6: icmp_seq=3 ttl=64 time=0.065 ms
64 bytes from 10.9.0.6: icmp_seq=4 ttl=64 time=0.067 ms
64 bytes from 10.9.0.6: icmp_seq=5 ttl=64 time=0.063 ms
64 bytes from 10.9.0.6: icmp_seq=6 ttl=64 time=0.049 ms
```

Photo #3: Script output on host A

```

Source IP:      10.9.0.5
Destination IP: 10.9.0.6
Protocol:      1
-----
Source IP:      10.9.0.6
Destination IP: 10.9.0.5
Protocol:      1
-----
Source IP:      10.9.0.5
Destination IP: 10.9.0.6
Protocol:      1
-----
Source IP:      10.9.0.6
Destination IP: 10.9.0.5
Protocol:      1
-----
Source IP:      10.9.0.5
Destination IP: 10.9.0.6
Protocol:      1
-----
Source IP:      10.9.0.6
Destination IP: 10.9.0.5
Protocol:      1
-----
Source IP:      10.9.0.5
Destination IP: 10.9.0.6
Protocol:      1
-----
Source IP:      10.9.0.6
Destination IP: 10.9.0.5
Protocol:      1
-----
Source IP:      10.9.0.5
Destination IP: 10.9.0.6
Protocol:      1
-----

```

Photo #4: Ping output on host B

Running the same script as a normal user, without sudo, failed immediately with a permission error when scapy tried to open the raw socket on the interface.

```

ubuntu@ip-172-31-47-241:~$ python3 sniffer.py
Traceback (most recent call last):
  File "/home/ubuntu/sniffer.py", line 10, in <module>
    pkt = sniff(iface='br-aaaa556e9e85', filter='icmp', prn=print_pkt)
  File "/usr/lib/python3/dist-packages/scapy/sendrecv.py", line 1036, in sniff
    sniffer._run(*args, **kwargs)
  File "/usr/lib/python3/dist-packages/scapy/sendrecv.py", line 906, in _run
    sniff_sockets[L2socket(type=ETH_P_ALL, iface=iface,
  File "/usr/lib/python3/dist-packages/scapy/arch/linux.py", line 410, in __
init__
    self.ins = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.htons
(type)) # noqa: E501
  File "/usr/lib/python3.10/socket.py", line 232, in __init__
    _socket.socket.__init__(self, family, type, proto, fileno)
PermissionError: [Errno 1] Operation not permitted

```

Photo #5: Running script without sudo

Packet sniffing requires opening a raw packet socket, which lets a process receive every frame the kernel delivers to the NIC, regardless of destination. Creating that socket requires the `CAP_NET_RAW` capability, which is only granted to the root by default. A regular user doesn't have `CAP_NET_RAW`, so the kernel refuses the `socket()` call before any packets can be captured, that's when the script fails when run without sudo.

Task 1.1B

I tested the three required filters by editing the filter argument in sniffer.py and re-running the program with sudo for each one.

Filter 1: Only ICMP packets

Test traffic: ping 10.9.0.6 from inside hostA. The sniffer captured both echo requests and echo replies as seen in Task 1.1A.

(See photos #3 and #4)

Filter 2: TCP from an IP, destination port 23

filter='tcp and src host 10.9.0.5 and dst port 23'

Test traffic: telnet 10.9.0.6 from inside host A. host B already had a telnet daemon listening, so the connection was established. The sniffer captured the TCP packets matching the filter (source 10.9.0.5, destination port 23).

```
#!/usr/bin/env python3
from scapy.all import *

def print_pkt(pkt):
    print("Source IP:      ", pkt[IP].src)
    print("Destination IP: ", pkt[IP].dst)
    print("Protocol:         ", pkt[IP].proto)
    print("-" * 40)

pkt = sniff(iface='br-aeaa556e9e85', filter='tcp and src host 10.9.0.5 and dst port 23',
, prn=print_pkt)
```



Photo #6: Script with filter variable changed

```

ubuntu@ip-172-31-47-241:~$ sudo python3 sniffer.py
Source IP:      10.9.0.5
Destination IP: 10.9.0.6
Protocol:       6
-----
Source IP:      10.9.0.5
Destination IP: 10.9.0.6
Protocol:       6
-----
Source IP:      10.9.0.5
Destination IP: 10.9.0.6
Protocol:       6
-----
Source IP:      10.9.0.5
Destination IP: 10.9.0.6
Protocol:       6
-----
Source IP:      10.9.0.5
Destination IP: 10.9.0.6
Protocol:       6
-----
Source IP:      10.9.0.5
Destination IP: 10.9.0.6
Protocol:       6
-----
Source IP:      10.9.0.5
Destination IP: 10.9.0.6
Protocol:       6
-----
Source IP:      10.9.0.5
Destination IP: 10.9.0.6
Protocol:       6
-----

```

Photo #7: Output on host B

Filter 3: Packets to/from a subnet

filter='net 128.230.0.0/16'

Test traffic: ping -c 3 128.230.18.63 (Syracuse University address) from inside host A. The sniffer captured the outgoing ICMP echo requests addresses to that subnet.

```

#!/usr/bin/env python3
from scapy.all import *

def print_pkt(pkt):
    print("Source IP:      ", pkt[IP].src)
    print("Destination IP: ", pkt[IP].dst)
    print("Protocol:       ", pkt[IP].proto)
    print("-" * 40)

pkt = sniff(iface='br-aeaa556e9e85', filter='net 128.230.0.0/16',
            prn=print_pkt)

```

Photo #8: filter variable changed to different ip


```

[ubuntu@ip-172-31-47-241:~]$ sudo python3 sniffer.py
Source IP:      10.9.0.5
Destination IP: 128.230.18.63
Protocol:      1
-----
Source IP:      128.230.18.63
Destination IP: 10.9.0.5
Protocol:      1
-----
Source IP:      10.9.0.5
Destination IP: 128.230.18.63
Protocol:      1
-----
Source IP:      128.230.18.63
Destination IP: 10.9.0.5
Protocol:      1
-----
Source IP:      10.9.0.5
Destination IP: 128.230.18.63
Protocol:      1
-----
Source IP:      128.230.18.63
Destination IP: 10.9.0.5
Protocol:      1
-----

```

Photo #9: Output on host B

Task 1.2: Spoofing ICMP Packets

I wrote a Scapy script that constructs an ICMP echo request with a forged source IP address and sends it to host A. Since I had issues booting the SEED VM, I used tcpdump on the bridge interface in place of Wireshark to capture.

```

#!/usr/bin/env python3
from scapy.all import *

ip = IP(src='1.2.3.4', dst='10.9.0.5')
icmp = ICMP()
pkt = ip/icmp

pkt.show()
send(pkt, verbose=1)

```

Photo #10: spoof.py script

In one terminal I ran tcpdump on the bridge interface filtering for ICMP:

```
sudo tcpdump -i br-aaaa556e9e85 -n icmp
```

In a second terminal I ran the spoof script, tcpdump captured the spoofed echo request and the echo reply that host A generated in response.

```
ubuntu@ip-172-31-47-241:~$ sudo python3 spoof_icmp.py
###[ IP ]###
version    = 4
ihl        = None
tos        = 0x0
len        = None
id         = 1
flags      =
frag       = 0
ttl        = 64
proto      = icmp
chksum     = None
src        = 1.2.3.4
dst        = 10.9.0.5
\options   \
###[ ICMP ]###
type       = echo-request
code       = 0
chksum     = None
id         = 0x0
seq        = 0x0
.
Sent 1 packets.
```

Photo #11: Running spoof script with sudo

```
ubuntu@ip-172-31-47-241:~$ sudo tcpdump -i br-aaaa556e9e85 -n icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on br-aaaa556e9e85, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C
0 packets captured
0 packets received by filter
0 packets dropped by kernel
[ubuntu@ip-172-31-47-241:~$ sudo tcpdump -i br-aaaa556e9e85 -n icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on br-aaaa556e9e85, link-type EN10MB (Ethernet), snapshot length 262144 bytes
02:19:51.130785 IP 1.2.3.4 > 10.9.0.5: ICMP echo request, id 0, seq 0, length 8
02:19:51.130820 IP 10.9.0.5 > 1.2.3.4: ICMP echo reply, id 0, seq 0, length 8
```

Photo #12: Output on host A

Host A had no way to know that the source IP was forged. It processed the ICMP echo request as it would any other and generated an echo reply addressed back to 1.2.3.4. The reply going to a non-existent address is the proof that the spoofing worked. If host A were checking source legitimacy it would have dropped the request, but standard IP/ICMP processing has no check. The reply is just dropped on the LAN since no machine owns 1.2.3.4, but tcpdump on the bridge sees both packets.

Task 1.3: Traceroute

I wrote a Scapy traceroute that sends ICMP echo requests with increasing TTL values, starting at 1, and records the source address of each ICMP Time-Exceeded reply. When an actual echo reply (0) comes back instead of a Time-Exceeded message, the destination was reached and the loop exits.

```
#!/usr/bin/env python3
from scapy.all import *
import sys

target = sys.argv[1] if len(sys.argv) > 1 else '8.8.8.8'

print(f"Tracing route to {target}\n")
for ttl in range(1, 31):
    pkt = IP(dst=target, ttl=ttl) / ICMP()
    reply = sr1(pkt, verbose=0, timeout=2)
    if reply is None:
        print(f"{ttl:2} * (no reply)")
    elif reply.type == 0:
        print(f"{ttl:2} {reply.src} (destination reached)")
        break
    else:
        print(f"{ttl:2} {reply.src}")
```

Photo #13: tracroute.py script

Tracing to 8.8.8.8 from my AWS instance reached the destination in a small number of hops. I also tested www.syracuse.edu which had 7 hops, and www.mit.edu with 10 hops.

```
ubuntu@ip-172-31-47-241:~$ sudo python3 traceroute.py 8.8.8.8
Tracing route to 8.8.8.8

 1  242.5.19.3
 2  151.148.13.60
 3  151.148.13.61
 4  142.251.226.65
 5  216.239.51.117
 6  8.8.8.8 (destination reached)
```

Photo #14: Testing script on 8.8.8.8

```
ubuntu@ip-172-31-47-241:~$ sudo python3 traceroute.py www.syracuse.edu
Tracing route to www.syracuse.edu

 1  100.64.88.167
 2  108.166.252.42
 3  240.3.104.13
 4  240.3.104.18
 5  240.3.104.21
 6  240.64.194.202
 7  3.170.131.49 (destination reached)
```


Photo #15: Testing script on syracuse website

```
ubuntu@ip-172-31-47-241:~$ sudo python3 traceroute.py www.mit.edu
Tracing route to www.mit.edu

 1  242.4.178.193
 2  99.82.13.184
 3  99.82.13.185
 4  23.209.174.70
 5  23.207.225.89
 6  23.203.151.173
 7  * (no reply)
 8  * (no reply)
 9  * (no reply)
10 23.194.149.213 (destination reached)
```

Photo #16: Testing script on mit.edu

Each router on the path decrements the TTL of a IP packet by one. When TTL reaches zero the router drops the packet and returns an ICMP Time-Exceeded message to the original sender, which reveals that router's IP. By starting with TTL = 1 and incrementing, you walk through the path one router at a time. When the packet finally reaches the destination, it produces an ICMP echo reply instead. Lines marked ' * ' are routers that exist on the path but are configured to silently drop ICMP Time-Exceeded responses. This is more of an estimate rather than a guaranteed path since each probe could be routed differently. Since I ran this from a AWS EC2 instance, the first few hops are inside Amazon's data center network rather than the public internet and they show up as private/CGNAT-style addresses. The later hops are real internet routers leading to the destination.

Task 1.4: Sniffing and-then Spoofing

I wrote a sniff-and-spoof program that listens on the bridge interface for ICMP echo requests. When it sees one, it constructs a forged echo reply with the source and destination IP's swapped, copies the original id, sequence number, and payload and sends the reply back out on the same interface. This makes every ping from host A appear to succeed regardless of whether the target IP is alive.

```

#!/usr/bin/env python3
from scapy.all import *

IFACE = 'br-aeaa556e9e85' #bridge interface

def spoof_reply(pkt):
    if ICMP in pkt and pkt[ICMP].type == 8:
        print(f"Sniffed echo request: {pkt[IP].src} -> {pkt[IP].dst}")

        ip = IP(src=pkt[IP].dst, dst=pkt[IP].src)
        icmp = ICMP(type=0, id=pkt[ICMP].id, seq=pkt[ICMP].seq)
        data = pkt[Raw].load if Raw in pkt else b''
        reply = ip/icmp/data

        send(reply, iface=IFACE, verbose=0)
        print(f"    -> spoofed reply sent as {ip.src}\n")

print(f"Sniffing on {IFACE}...")
sniff(iface=IFACE, filter='icmp', prn=spoof_reply)

```

Photo #17: sniff-and-spoof.py script

With the program running on the host, I pinged each of the three IPs from inside the ost container.

Case 1: ping 1.2.3.4

ping reports replies coming back from 1.2.3.4 with ttl=64, even though that host doesn't exist. The sniff-and-spoof program logged that it captured the echo requests and sent back spoofed replies.

```

root@0910a03e5e64:/# ping 1.2.3.4
PING 1.2.3.4 (1.2.3.4) 56(84) bytes of data.
64 bytes from 1.2.3.4: icmp_seq=1 ttl=64 time=24.3 ms
64 bytes from 1.2.3.4: icmp_seq=2 ttl=64 time=7.67 ms
64 bytes from 1.2.3.4: icmp_seq=3 ttl=64 time=7.69 ms
64 bytes from 1.2.3.4: icmp_seq=4 ttl=64 time=7.91 ms
64 bytes from 1.2.3.4: icmp_seq=5 ttl=64 time=7.81 ms
64 bytes from 1.2.3.4: icmp_seq=6 ttl=64 time=8.13 ms
64 bytes from 1.2.3.4: icmp_seq=7 ttl=64 time=8.31 ms

```

Photo #18: Pinging 1.2.3.4

Host A's routing table sends traffic for 1.2.3.4 through its default gateway (the host machine). The echo request goes out on the bridge interface, where the sniffer sees it and immediately forges a matching echo reply. The ttl=64 in the reply shows that the response was generated on the local network rather than from across the internet.

Case 2: ping 10.9.0.99

ping prints 'From 10.9.0.5 icmp_seq=N Destination Host Unreachable'. The sniff-and-spoof script captured nothing for this destination.

```

root@0910a03e5e64:/# ping 10.9.0.99
PING 10.9.0.99 (10.9.0.99) 56(84) bytes of data.
From 10.9.0.5 icmp_seq=1 Destination Host Unreachable
From 10.9.0.5 icmp_seq=2 Destination Host Unreachable
From 10.9.0.5 icmp_seq=3 Destination Host Unreachable
From 10.9.0.5 icmp_seq=4 Destination Host Unreachable
ping: sendmsg: No route to host
From 10.9.0.5 icmp_seq=5 Destination Host Unreachable
From 10.9.0.5 icmp_seq=6 Destination Host Unreachable

```

Photo #19: Pinging 10.9.0.99

10.9.0.99 is on host A's local subnet (10.9.0.0/24). Before sending an IP packet to a host on the same subnet, host A must first resolve that host's MAC address using ARP. It broadcasts an ARP request 'who has 10.9.0.99?' but no machine answers because that host does not exist. With no MAC address to send to, the ICMP echo request is never put on the wire at all, so the sniffer never sees it and has nothing to spoof a reply for. The 'Destination Host Unreachable' message is generated locally by host A's own kernel after the ARP timeout.

Case 3: ping 8.8.8.8

ping receives two replies for every echo request. One has ttl=115 (the genuine reply from Google's 8.8.8.8 server, with TTL decremented by all the Internet routers on the path), and the other has ttl=64 marked as DUP! (my spoofed reply, generated locally so its TTL is still the default 64).

```

root@0910a03e5e64:/# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=9.00 ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=64 time=33.4 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=9.06 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=64 time=13.9 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=3 ttl=64 time=7.61 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=115 time=8.96 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=4 ttl=64 time=6.48 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=115 time=8.96 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=5 ttl=64 time=8.52 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=115 time=8.88 ms (DUP!)

```

Photo #20: Pinging 8.8.8.8

The echo request is routed via the default gateway out to the real internet, but the sniffer also sees it as it crosses the bridge interface and forges a reply. The real reply eventually comes back from 8.8.8.8 as well. Linux ping accepts the first reply that matches a given (id,seq) and marks any subsequent matching replies as duplicates. The two TTL values are the clearest evidence that one reply is locally generated and the other really did traverse the internet

Task Set 2 is not required in this Lab.